

# An Incremental Inverse Reinforcement Learning Approach for Motion Planning with Separated Path and Velocity Preferences

Armin Avaei<sup>†,1</sup>, Linda van der Spaa<sup>†,1,2,\*</sup>, Luka Peternel<sup>1</sup> and Jens Kober<sup>1</sup>

**Abstract**—Humans often demonstrate diverse behaviors due to their personal preferences, for instance, related to their individual execution style or personal margin for safety. In this paper, we consider the problem of integrating both path and velocity preferences into trajectory planning for robotic manipulators. We first learn reward functions that represent the user path and velocity preferences from kinesthetic demonstration. We then optimize the trajectory in two steps: first the path and then the velocity, to produce trajectories that adhere to both task requirements and user preferences. We design a set of parameterized features that capture the fundamental preferences in a pick-and-place type of object-transportation task, both in shape and timing of the motion. We demonstrate that our method is capable of generalizing such preferences to new scenarios. We implement our algorithm on a Franka Emika 7-DoF robot arm, and validate the functionality and flexibility of our approach in a user study. The results show that non-expert users are able to teach the robot their preferences with just a few iterations of feedback.

**Index Terms**—Learning from demonstration, Human preferences, Incremental inverse reinforcement learning, Coactive learning, Physical human-robot interaction

## I. INTRODUCTION

Autonomy is increasingly being discussed under the aspect of cooperation. A gentler breed of robots, “cobots”, have started to appear in factories, workshops and construction sites, working together with humans. A challenge in the deployment of such robots is producing desirable trajectories for object-carrying tasks. A desirable trajectory not only meets the task constraints (e.g. collision-free movement from start to goal), but also adheres to user preferences. Such preferences may vary between users, environments and tasks. It is infeasible to manually encode them without exact knowledge of how, with whom, and where the robot is being deployed (Jain et al., 2015). Manual programming is even more detrimental in cooperative environments, where robots are required to be easily and rapidly reprogrammed. In this context, learning preferences directly from humans emerges as an attractive solution.

We address the challenge of learning personalized human preferences, starting from a robot plan that may not match the execution style or safety standards of a specific human user (e.g. robot carries the object closer to the obstacle than the

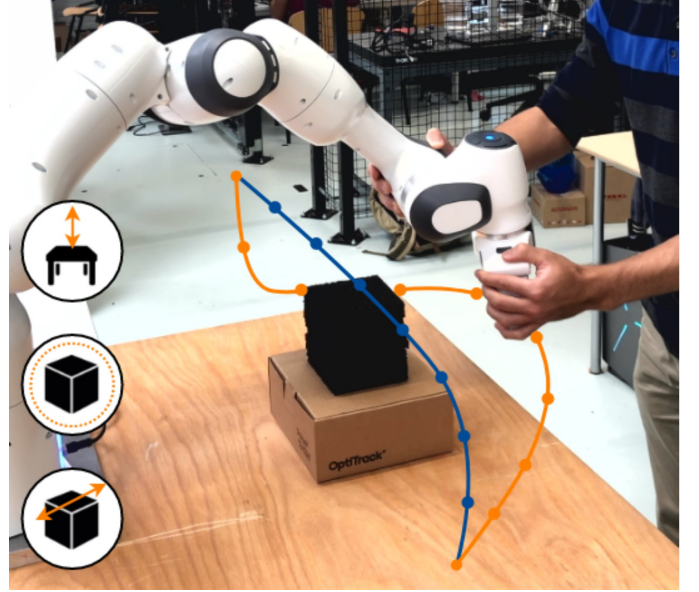


Fig. 1. Leveraging demonstrations as means of understanding the human’s preferences in an object-carrying task: The robot originally plans the blue trajectory without knowledge of human preferences. The user demonstrates the orange trajectory which in this instance contains the following preferences: “Stay close to the table surface”, “Keep larger distance from the obstacle”, and “Pass on the far side of the obstacle”. We develop a method for learning and generalizing such preferences to new scenarios (i.e. new start, goal or obstacle positions).

user prefers). Fig. 1 illustrates how a user may demonstrate a trajectory encoding multiple implicit preferences to correct the original robot plan.

One way to adhere to human preferences is by means of variable impedance control (Duchaine and Gosselin, 2007; Peternel et al., 2014). While such strategies can ensure safe and responsive adaptation, they suffer from being purely reactive (i.e. they do not remember the corrections). The robot should not only conform to a new trajectory, but it has to update its internal model in order to understand the improvement of the corrected trajectory (Bajcsy et al., 2017; Losey and O’Malley, 2019; Losey et al., 2022). Thus, ideally, we should encode knowledge of humans’ desired trajectories as a set of parameters that are incrementally updated based on the corrected trajectory.

To this end, Learning from Demonstration (LfD) approach enables robots to encode human-demonstrated trajectories. LfD frameworks have the advantage of enabling non-experts to

<sup>†</sup>These authors contributed equally to this work

<sup>1</sup>Department of Cognitive Robotics, Delft University of Technology, Delft, The Netherlands

<sup>2</sup>Honda Research Institute Europe, Offenbach/Main, Germany

\*Corresponding author, email: l.f.vanderspaa@tudelft.nl

naturally teach trajectories to robots. A widespread trajectory learning method in LfD is Dynamic Movement Primitives (DMPs) (Ijspeert et al., 2002). In addition to encoding trajectories, DMPs are able to adapt the learned path by updating an interactive term in the model (Kulvicius et al., 2013; Gams et al., 2016). Additionally, they can adapt the velocity of the motion by estimating the frequency and the phase of a periodic task (Peternel et al., 2014), or learning a speed scaling factor (Nemec et al., 2018). As a result, DMPs can capture human path and velocity preferences on a trajectory level. Losey and O’Malley (2019) demonstrated that such velocity preferences can also be learned online, from interactive feedback, although with some effort. However, these methods lack any knowledge about the task context or why the trajectory was adjusted in the first place. Hence, such an approach fails to generalize user preferences to new scenarios due to the lack of a higher-level understanding of human actions.

A better approach is to pair parameters with features that capture contextual information (e.g. distance to obstacle), and utilizes this information to find an optimal solution in new scenarios. Such generalization can be achieved by learning a model of what makes a trajectory desirable. Modeling assumptions can be made to form a conditional probability distribution over trajectories and contextual information, e.g. as in Ewerton et al. (2016). While proven effective in simple reaching tasks, whether such models can directly capture complex human preferences in a contextually rich environment remains an open question. However, Inverse Reinforcement Learning (IRL) approaches have already proven to be capable of this (Wirth et al., 2017).

Unlike traditional IRL methods requiring expert demonstrations (Ratliff et al., 2006; Ziebart et al., 2008), more recently derived algorithms allow preference learning from user comparisons of sub-optimal trajectories (Wirth et al., 2017). Potentially, a much wider range of human behavior can be interpreted as feedback for preference learning in general (Jeon et al., 2020). In this paper, however, we focus on reward learning for robot trajectories. A model-free approach can be used to learn complex non-linear reward functions (Ibarz et al., 2018), but such an approach requires many queries to learn from, which is time-intensive. Therefore, we keep a simple linear reward structure. To shape this reward, we identified four fundamental preference features of the pick-and-place type of object-transportation tasks in the literature: height from table/ground (Jain et al., 2015; Bajcsy et al., 2017; Losey et al., 2022), distance to obstacle (Jain et al., 2015; Bıyık et al., 2022), obstacle side (Kirby et al., 2009; Kretschmar et al., 2016), and velocity (Peternel et al., 2014; Nemec et al., 2018). These features are relatively scenario-unspecific, and therefore suitable for generalization in object-transportation tasks of the kind we consider in this paper: pick-and-place tasks in the presence of obstacles. To the best of our knowledge, there is no method to account for all these features together in a unified framework.

Given such a set of features, coactive learning (Shivaswamy and Joachims, 2015) can be used to learn a reward function. In coactive learning, the learner and the teacher both play an active role in the learning process: the learner proposes one or

multiple solutions and learns from relative feedback provided by the teacher in response. Coactive learning has an upper boundary on regret, leaving room for noisy and imperfect user feedback. Furthermore, it is an online algorithm, i.e., the system can learn incrementally from sequential feedback. An adapted version of coactive learning was applied in Jain et al. (2015) to learn trajectory preferences in object-carrying tasks. To this end, users iteratively ranked trajectories proposed by the system. Although selected based on the learned reward, the trajectories were generated using randomized sampling, which increases the number of feedback iterations necessary for convergence. Methods in Bajcsy et al. (2017); Losey et al. (2022) adapt the robot trajectory to a user’s preferences based on force feedback and optimize the remaining trajectory with online correction in a specific scenario. However, these methods cannot capture velocity preferences on top of path preferences.

To address this gap in the state-of-the-art, we propose a novel framework for optimizing trajectories in object-transportation tasks that meet the user’s path and velocity preferences, where we first optimize the path and then the velocity on the path. The objective function for the optimization comprises a human preferences reward function and a robot objective function that ensures the safety and efficiency of the trajectories. This explicit separation of the agents’ objectives allows for negotiation, where the robot is recognized as an intelligent agent which may give valuable input of its own.

The approach takes a full demonstrated trajectory as the feedback for the learning model, comparing it to the robot’s previous plan at each iteration. A minimum acceleration trajectory model significantly reduces the size of the task space, hence increasing the optimization efficiency. To capture the preferences, we design a set of features that correspond to the four preferences, covering both the motion shape and timing, which we identified from the literature to be fundamental for the considered pick-and-place tasks.

Unlike Bajcsy et al. (2017); Losey et al. (2022), we request iterative feedback and employ an optimization scheme that samples from the global trajectory space. While this is less efficient in terms of human effort for teaching preferences in a specific scenario (i.e. the user has to provide at least one full task demonstration), it allows us to additionally capture velocity preferences on top of the path preferences. Furthermore, our method enables the separation of velocity and path preferences both during the learning and in the trajectory optimization stage. With our combination of a trajectory optimization scheme and carefully selected preference features, we can generalize to new contexts without needing (many) additional corrective demonstrations. In contrast to Jain et al. (2015), we learn from a few informative feedback demonstrations, and give special attention to the trajectory sampling by employing model-based trajectory optimization. This facilitates fast learning and generalization of preferences to entirely new contexts.

We evaluate the proposed method in a user study on a 7-DoF Franka Emika robot arm. In the key previous user studies of learning human preferences (Palan et al., 2019; Bajcsy et al., 2017; Losey et al., 2022), the experimenter instructed

the human participants what preference to demonstrate to the robot. Differently, in our user study, we let the participants freely select their own preferences while demonstrating the task execution to the robot. Additionally, our study examines whether the users can actually distinguish the learned trajectory capturing their preference from the trajectories capturing only part of their preference. In a supplementary study, we qualitatively compare our method to two relevant methods from the literature. We discuss the structural differences between the methods, and show in simulation how these differences affect the learning of preferences from human (corrective) demonstrations.

In summary, this paper’s main contribution is a methodology that is able to capture velocity preferences on top of path preferences by separating the velocity optimization from the path optimization. Learning the path and velocity separately provides users with the option to avoid the challenge of providing a temporally consistent demonstration at each iteration. This offers users the flexibility to demonstrate their path and velocity preferences either simultaneously or in separate demonstrations. Secondly, the learned preferences are transferred to new scenarios by exploiting a trajectory model. Importantly, we perform a user study to validate whether the proposed method can learn and generalize freely chosen preferences, in contrast to the many user studies in the literature which prescribe user preferences. Additionally, we perform a supplementary study to compare pros and cons of the proposed approach to two common methods from the literature.

The rest of the paper is organized as follows: In Sec. II, we explain the algorithm and methodology in detail. The user study is described in Sec. III, and the experimental results are shown and discussed. A supplementary study is presented and discussed in Sec. IV. Finally, we present our conclusion and view on future work in Sec. V.

## II. METHOD

The problem is defined in the following manner: given a context  $\mathcal{C}$  describing start, goal, and obstacle positions, the robot has to determine the trajectory  $\xi = [\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_N] \in \Xi$  (set of state sequences) that conforms to the human preferences and meets the task goals. The states are defined as  $\mathbf{s}_k = [\mathbf{x}_k; \dot{\mathbf{x}}_k]$  (position and velocity), with  $k$  indicating trajectory samples.

In our setting, the true reward functions are known by the user but not directly observable by the robot. Hence the problem can be seen as a Partially Observable Markov Decision Process (POMDP) (Bajcsy et al., 2017). Our reward functions have parameters that are part of the hidden state, and the trajectories provided by the user are observations about these parameters. Solving such problems, where the control space is very complex and high-dimensional, is challenging. Therefore, we simplify the problem through approximation of the policy by separating planning and control, and treating it as an optimization problem. Furthermore, we make the problem tractable by reducing our state space to one of viable smooth trajectories.

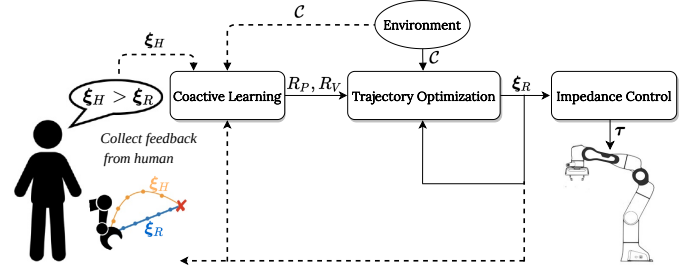


Fig. 2. The human user provides demonstrations, which are used to learn a distribution over reward functions via coactive learning. We use the learned rewards to optimize the robot’s trajectory according to the human preferences. The resulting trajectory is executed using an impedance controller. We repeat this process, querying the human for preferred trajectories until convergence. The human can then be taken out of the loop.

The resulting framework, depicted in Fig. 2, first learns the appropriate reward functions, then plans a trajectory maximizing the rewards via optimization. Once the trajectory is defined, we use impedance control to track it in a safe manner. Notably, we separate the problem of path and velocity planning in the learning and optimization steps. Updating the path and velocity weights separately provides users with the option to avoid the challenge of providing a temporally consistent demonstration at each iteration. As a result, users have the flexibility to demonstrate their path and velocity preferences either simultaneously or in separate demonstrations.

### A. Learning human reward functions from demonstration

We follow previous IRL work (Jain et al., 2015; Ratliff et al., 2006) in assuming that the reward functions are a linear combination of features  $\phi$  with weights  $\theta$ . Accordingly, we define path and velocity reward functions  $R_P$  and  $R_V$  as

$$R_P(\mathbf{x}; \mathcal{C}, \theta_{HP}) = \theta_{HP}^T \Phi_P(\mathbf{x}; \mathcal{C}), \quad (1a)$$

$$R_V(\bar{\mathbf{x}}, \bar{\mathbf{x}}; \mathcal{C}, \theta_{HV}) = \theta_{HV}^T \Phi_V(\bar{\mathbf{x}}, \bar{\mathbf{x}}; \mathcal{C}), \quad (1b)$$

where  $\theta_{HP}$  and  $\theta_{HV}$  denote the unknown weights that respectively capture the human path and velocity preferences. In case of the velocity reward, we divide the trajectory into equal segments (i.e. range of samples) indicated by  $r$ . Then,  $\bar{\mathbf{x}}_r$  and  $\bar{\dot{\mathbf{x}}}_r$  are the average of the position vectors and the velocity norms in a segment.  $\Phi_P$  and  $\Phi_V$  are the total path and velocity feature counts along the trajectory:

$$\Phi_P(\mathbf{x}; \mathcal{C}) = \sum_{k=1}^N \phi_P(\mathbf{x}_k; \mathcal{C}), \quad \Phi_V(\bar{\mathbf{x}}, \bar{\mathbf{x}}; \mathcal{C}) = \sum_{r=1}^M \phi_V(\bar{\mathbf{x}}_r, \bar{\dot{\mathbf{x}}}_r; \mathcal{C}). \quad (2)$$

Note that the velocity features are a function of both the segment’s velocity and position, allowing us to capture position-dependent velocity preferences.

To have comparable rewards, all trajectories are re-sampled to contain a fixed number of  $N$  states. The velocity inherently affects the number of samples within a trajectory, which is why we divide the trajectory into  $M$  segments and consider the average velocity within each segment ( $M < N$ ). Features are directly computed from the robot state and context of the task. We describe them in the next subsection.

During kinesthetic demonstration, the robot is in gravity compensation mode. That gives the human full control over the demonstrated trajectories, which we assume to correlate exponentially to the human's internal reward:

$$P(\xi_H | \mathcal{C}, \theta_{HP}, \theta_{HV}) \propto e^{\theta_{HP}^T \Phi_P(\xi_H; \mathcal{C}) + \theta_{HV}^T \Phi_V(\xi_H; \mathcal{C})}, \quad (3)$$

which, for brevity, we can write as  $P(\xi_H | \mathcal{C}, \theta_H) \propto e^{\theta_H^T \Phi(\xi_H; \mathcal{C})}$ .

Assuming that the human behavior is approximately optimal with respect to the true reward (i.e. their preferences), we use a variant of coactive learning introduced in Bajcsy et al. (2017) to learn the weights  $\theta_{HP}, \theta_{HV}$ . However, we can only compute our  $\Phi_P, \Phi_V$  (2) over full trajectories. Therefore, instead of updating the weights based on an estimate of the human's intended trajectory from physical interaction, we use a full kinesthetic trajectory demonstration by the human after each task execution to update the sum of the features over the trajectory (2). This results in the following incremental update rule:

$$\theta_H^{i+1} = \theta_H^i + \alpha (\Phi(\xi_H^i; \mathcal{C}) - \Phi(\xi_R^i; \mathcal{C})), \quad (4)$$

at iteration  $i$ , with learning rate  $\alpha \in (0, 1]$ . Intuitively, the update rule is a gradient that shifts the weights in the direction of the human's observed feature count. It should be noted that we update the path preferences only using the position part of the state, and the velocity preferences are updated depending on where in space the velocities were observed.

## B. Features and rewards

We define the objective function for trajectory optimization as a combination of human rewards and robot objectives. The human rewards consist of features that capture human preferences (1), whereas the robot objectives define a basic behavior for the robot. Moreover, the robot objectives counterbalance the effect of the human rewards in the optimization. While we learn the weights in the human rewards (Sec. II-A). The weights in the robot objectives are hand-tuned. In this section, we first describe the features associated with the human rewards, and then the robot objectives.

The human preferences are captured via the four features listed below (see Fig. 1 for an example of the listed path preferences). We chose these features as they characterize dominant behaviors in manipulation applications that depend on user preferences. Additionally, the features cover the different dimensions of the workspace (in space and time), creating a complete definition of motion behavior.

*Height from the Table:* The preferred height from the table, on a range of 'low' to 'high' is captured by the sigmoid function  $\phi_h = \frac{1}{1 + e^{-\lambda(h+p)}}$  with  $h$  indicating the vertical distance from the table,  $p$  the center of the function (an arbitrary 'medium' height above the table), and  $\lambda$  the parameter defining the shape of the function. The choice of a sigmoid function is to hinder the effect of this preference when close to upper and lower boundaries during the weight update (e.g., a demonstration at 75 cm above the table should not impact the weight update very differently from a demonstration at 70 cm). The decreasing slope at the boundaries additionally allows

other objectives to have a higher impact on the trajectory in such regions during the optimization.

*Distance to the Obstacle:* We encode the user's preferred distance to the obstacle, on a range of 'close' to 'far' using the exponential feature  $\phi_d = e^{-\beta d^2}$ , where  $d$  is the Euclidean distance to the center of the obstacle, and  $\beta$  is the shape parameter. This exponential function gradually drops to 0 at a certain distance from the obstacle. This distance is a threshold outside which the local behavior of the optimization is no longer affected by the distance to the obstacle. Importantly, if a negative weight is learned associated with this feature, the trajectory is still attracted towards the obstacle even if the initial trajectory lies outside of this threshold. This is because our optimization strategy globally explores different regions of the workspace, and in this case it would detect that there is a reward associated with being closer to the obstacle.

*Obstacle Side:* We define this feature on a range of 'close' (the side of the obstacle closer to the robot) to 'far' (the side of the obstacle far from the robot) via the tangent hyperbolic function  $\phi_s = \frac{2}{1 + e^{\gamma s}} - 1$ . Here,  $S$  is the lateral distance between a trajectory sample and the vertical plane at the center of the obstacle, and  $\gamma$  is a shape parameter. This symmetric function is designed to have a large span in order to be active in all regions of the workspace. However, as the gradient of this function decreases at larger lateral distances, so does the influence of this function in the local trajectory optimization.

*Velocity:* To encapsulate the user's velocity preferences, we adopt a different approach using a discretized linear combination of uniformly distributed Radial Basis Functions (RBFs) in a range  $[\dot{x}_{\min}, \dot{x}_{\max}]$ . For each segment  $r$ , we map the average velocity norm  $\bar{\dot{x}}_r$  onto these RBFs, given by:

$$\psi_j(\bar{\dot{x}}_r) = e^{-(\varepsilon \bar{\dot{x}}_r - c_j)^2}, \quad (5)$$

where shape variable  $\varepsilon$  defines the width, and  $c_j$  defines the center of the  $j^{\text{th}}$  RBF, with  $j = 1, 2, \dots, n$  (we use  $n=9$ ).

Inspired by Fahad et al. (2018), we discretize the above feature to two bins, based on the distance  $d_r$  of each segment center to the obstacle. Hence, we have two cumulative feature vectors:  $\Phi_{V1}$  for  $d_r \in [0, d_c)$ , and  $\Phi_{V2}$  for  $d_r \in [d_c, \infty)$ . This allows us to approximate the speed of motion separately in areas considered to be respectively 'close' to or 'far' from the obstacle based on the distance threshold  $d_c$  (obtained from demonstration data). This way, we capture velocity preferences relative to the obstacle position. Similarly, features can be defined relative to other context parameters, to capture velocity preferences that depend on other parameterized positions.

However, the issue might arise that the two trajectories do not have the same number of segments in each distance bin. In such a case, we employ feature imputation using the mean of the available values.

The robot's objectives are composed of the following:

*Path Efficiency Reward:* We calculate the total length of a trajectory, which we use as a negative reward. Penalizing the trajectory length is essential in counterbalancing the human preference features in the optimization process. Essentially, it pulls the trajectories towards the straight line path from start to goal and rewards keeping them short.



**Collision Avoidance Reward:** We use the obstacle cost as formulated by Zucker et al. (2013), which increases exponentially once the distance to the obstacle drops below a threshold. The negative cost is our reward.

**Robot Velocity Reward:** This reward achieves a low and safe velocity in absence of human velocity preferences and is defined based on (5). In IRL, it is beneficial to learn how people balance other features against a default reward (Vasquez et al., 2014).

### C. Motion planning via trajectory optimization

We discuss the problem of motion planning in two parts. First, we address the optimization of the path of the trajectory in the workspace. We then address the optimization of the velocity along this path, defining the timing of the motion.

Solving the path optimization problem over the Cartesian task-space would be complex and inefficient. Instead, we employ a trajectory planning algorithm (MathWorks, 2018) that interpolates between waypoints with piecewise clothoid curves. This algorithm minimizes the acceleration which results in a smooth and realistic motion. We exploit this algorithm to significantly reduce the search space for the path optimization, and sample trajectories using a vector of waypoint coordinates  $\mathbf{p}$  and its corresponding time vector  $\mathbf{t}_P$ ,  $\xi = f(\mathbf{p}, \mathbf{t}_P)$ .

We consider three waypoints  $\mathbf{p} = [\mathbf{p}^s; \mathbf{p}^m; \mathbf{p}^g]$ , corresponding respectively to the start position, an arbitrary position within the path, and the goal position. We further simplify the problem by fixing the time vector to  $\mathbf{t}_P = t^g[0; \frac{D(\mathbf{p}^m)}{D(\mathbf{p}^g)}; 1]^T$ , where  $D(\cdot)$  indicates the Euclidean distance of a waypoint to  $\mathbf{p}^s$ , and  $t^g$  is the time, just for the path optimization, we assume all trajectories take to finish<sup>1</sup>. An uneven distribution of waypoints would bias the reward value. Setting up the time vector in this manner ensures a constant velocity throughout the trajectory, which results in an even distribution of samples over the path. Trajectories can then be sampled only as a function of waypoint positions  $\xi = f(\mathbf{p})$ .

We then solve for the optimal waypoint vector  $\mathbf{p}^*$  using the following non-linear program formulation:

$$\mathbf{p}^* = \arg \max_{\mathbf{p}} \left( R_P(\mathbf{p}; \mathcal{C}, \theta_{HP}) + \theta_{RP}^T \Phi_{RP}(\mathbf{p}; \mathcal{C}) \right), \quad (6)$$

subject to:  $\mathbf{h}(\mathbf{p}) = \mathbf{0}$ ,  $\mathbf{p}_{\text{low}} \leq \mathbf{p} \leq \mathbf{p}_{\text{upp}}$ .

Here, the objective function consists of the human path reward  $R_P$  and the robot's path objective, which is a linear combination of predetermined weights  $\theta_{RP}$  and the aforementioned path reward functions  $\Phi_{RP}$ . The equality constraint ensures the start and goal positions are met. As a result, we are effectively searching for the waypoint  $\mathbf{p}^m$  that maximizes the objective function. The upper and lower boundaries  $\mathbf{p}_{\text{low}}$  and  $\mathbf{p}_{\text{upp}}$  limit the trajectory to stay within the robot's workspace. Once  $\mathbf{p}^*$  is found, we construct the full trajectory using  $\xi_P^* = f(\mathbf{p}^*, \mathbf{t}_P)$ . Fig. 3 shows an example of the convergence of the optimizer towards a path that adheres to 'low height', 'close side' and 'close to obstacle' preferences.

<sup>1</sup>The shape of the paths is not affected by  $t^g$  in the time ranges of our manipulations, therefore we assume the path to be independent of velocity.

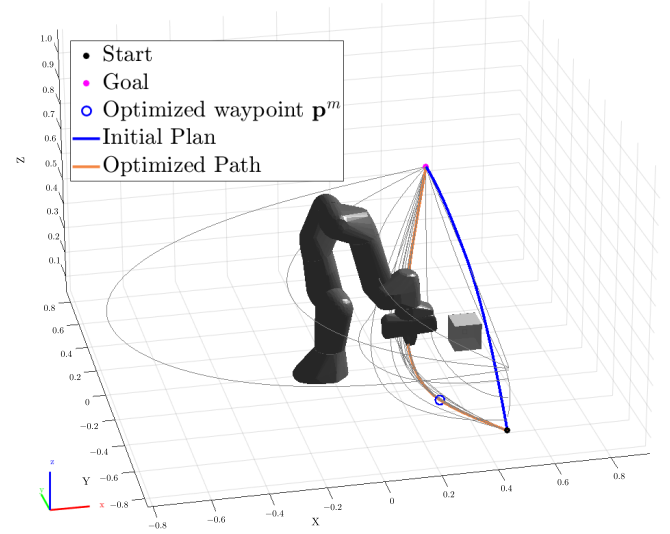


Fig. 3. An example of convergence towards the optimal path. The optimizer places  $\mathbf{p}^m$  in different locations in the workspace to generate different paths. The paths explored by the optimizer are indicated in gray. The orange path indicates the output of the path optimizer, resulting from placing the middle waypoint at the location indicated by the blue circle.

Having the optimal path  $\xi_P^*$ , we divide the trajectory into  $M$  segments (as described in Sec. II-A). Next, we store the positions of the waypoints at the end of the segments in  $\mathbf{p}_V^* = [\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_M]$ . This vector is fixed to maintain the shape of the trajectory. The corresponding timestamps, stored in  $\mathbf{t} = [t_1, t_2, \dots, t_M]$ , are the variables we optimize. Thus, trajectories sampled by the optimizer are only a function of the time vector  $\xi = f(\mathbf{t})$ . By optimizing  $\mathbf{t}$  we optimize the average velocity of each segment. The optimal time vector

$$\mathbf{t}^* = \arg \max_{\mathbf{t}} \left( R_V(\mathbf{t}; \mathcal{C}, \theta_{HV}) + \theta_{RV} \phi_{RV}(\mathbf{t}; \mathcal{C}) \right), \quad (7)$$

subject to:  $\mathbf{g}(\mathbf{t}) \leq \mathbf{0}$ ,  $\mathbf{t} \leq \mathbf{t}_{\text{upp}}$ ,

where the objective function is composed of  $R_V$  and the robot's velocity objective  $\phi_{RV}$ , which provides a reward for carrying objects at  $\dot{x}_{\text{robot}}$  with a fixed weight  $\theta_{RV}$ . The inequality constraint  $\mathbf{g}(\mathbf{t})$  bounds the velocity over each segment to  $\dot{x}_{\text{min}}$  and  $\dot{x}_{\text{max}}$ , not allowing the timestamps to get too close or far from each other. The upper boundary on  $\mathbf{t}$  acts as a limit on the total duration of motion.

Finally, the trajectory that adheres to both the path and velocity preferences is constructed using  $\xi_R = f(\mathbf{p}_V^*, \mathbf{t}^*)$ . The full method is summarized in Algorithm 1.

### III. METHOD VALIDATION WITH USER STUDY

To validate our framework we conduct two user experiments on a Franka Emika 7-DoF robot arm. Thereby we show a proof-of-concept of our approach in a real-world scenario with non-expert users. In both experiments, we use a set of three pick-and-place tasks in an agricultural setting as shown in Fig. 4. The primary goal of each task was moving the tomatoes from the initial position to the goal without any collisions with the obstacle. The experiments were approved by the

---

**Algorithm 1:** Learning human preferences from kinesthetic demonstration

---

```

1 Record  $\xi_H^0 = \{x_k, t_k\}_{k=1}^N$ , obtain context  $C$ 
2  $\dot{x}_k \leftarrow \frac{d}{dt}x_k$ , compute  $\dot{x}_r$  and  $\bar{x}_r$ 
3 Initialize  $\theta_H^0, \theta_R, \theta_R^0$ 
4 Set  $i = 0$ 
5 while executing task do
6   if Received Human Feedback then
7      $\theta_H^{i+1} = \theta_H^i + \alpha (\Phi(\xi_H^i; C) - \Phi(\xi_R^i; C))$ 
8    $p^* \leftarrow \text{Optimize}(\theta_{HP}^{i+1}, \theta_{RP}, C)$ 
9    $t^* \leftarrow \text{Optimize}(p^*, \theta_{HV}^{i+1}, \theta_{RV}, C)$ 
10   $\xi_R = f(p^*, t^*)$ 
11   $\tau \leftarrow \text{Impedance}(\xi_R)$ 
12   $i = i + 1$ 

```

---

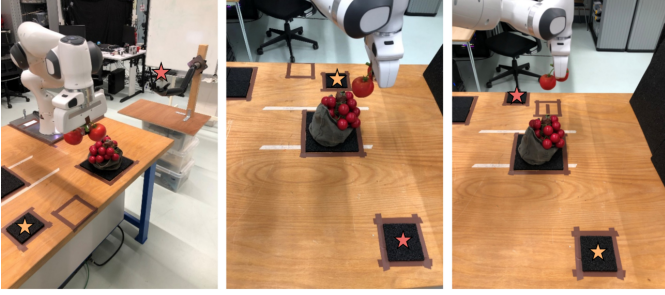


Fig. 4. From left to right Scenarios 1–3. The orange and red star, respectively, indicate the start and goal positions. The obstacle to be avoided is the bag of tomatoes. Scenario 1 and 2 shared the same starting positions, and Scenario 2 and 3 shared the same obstacle positions. Notice the difference in height of the goal position in Scenario 1 compared to Scenarios 2 and 3.

Human Research Ethics Committee at the Delft University of Technology on 06/09/2021.

We recruited 14 participants (4 women, 10 men) between 23 and 36 years old (mean = 26.8, SD = 3.6), six of whom had prior experience with robotic manipulators, but none of whom had any exposure to our framework.

Each user first took approximately 10 minutes to get familiar with physically manipulating the robot in the workspace. In this period, we also instructed users about the goal of the task and the preferences the robot could capture. Users then proceeded with the two experiments. To subjectively assess whether the framework can capture a range of different behaviors, in the first experiment we let the users freely choose their path and velocity preferences. Once users were more familiar with the framework, in the second experiment we assessed how effectively they could teach a set of pre-defined preferences to the robot. The overview of the user study is provided in Fig. 5. We discuss each experiment in the following subsections. A video of the experiments can be found here: <https://youtu.be/hhL5-Lpzj4M>.

#### A. User-Defined Preferences

In the first experiment, we investigate how our framework performs when users openly choose their set of preferences. We are specifically interested in assessing how well the robot

plans motions in new task instances with a context it has not seen before (i.e. generalization of preferences to new scenarios). We also evaluate the user experience in terms of acceptability and effort required from the user’s perspective. Accordingly, we test the following hypotheses:

**H1.** The proposed framework can capture and generalize user preferences to new task instances.

**H2.** Users feel a low level of interaction effort.

**Procedure and Measures.** Users first performed a demonstration in Scenario 1 (Fig. 4) for path preferences with the robot in gravity compensation mode. Notably, we did not limit users to a discrete set of preferences. For instance, instead of asking users to pass on either the close or far side of the obstacle, we asked them to intuitively demonstrate how far to either side of the obstacle they would prefer to pass. They could, for example, decide to pass right above the obstacle which would correspond to a “stay to the middle of the obstacle” for the “obstacle side” path preference. We then collected a second separate demonstration for the velocity preferences. During velocity demonstrations, the robot was only compliant along a straight line path covering the full range of distances to the obstacle. This allowed the users to demonstrate their preferred speed without having to care about the path. The velocity optimization step can take up to 3 minutes, therefore we simplified the method for learning and planning velocity preferences to find the velocity  $c_j$  with the highest feature count in this part of the study. Users were instructed to provide corrections via additional kinesthetic demonstrations (max 10 min per scenario) until they were satisfied with the resulting trajectory. However, the users were informed that the trajectory speed was only trained once and would not be updated further.

After observing each trajectory, the users filled out a subjective questionnaire for qualitative evaluation, rating the following statements on a 7-point Likert scale:

- 1) The robot accomplished the task well.
- 2) The robot understood my **path** preferences.
- 3) The robot understood my **motion** preferences.

To evaluate the effort, we counted the number of times a user-provided feedback, and let the participants fill out the NASA Task Load Index (TLX) at the end of this experiment. The independent variables of this experiment are the contexts which are varied for each scenario for assessing workload. While we do not compare results with a baseline here, NASA-TLX is still appropriate since it can capture absolute results (Hart and Staveland, 1988).

**Results.** Users demonstrated a multitude of path preferences, including “Keep low distance to the obstacle” and “Stay at medium height above the table”. Similarly, for velocity preferences, while the majority opted for a constant “medium” speed, both the preference of going “slower when close to the obstacle” and “faster when close to the obstacle” were demonstrated at least once.

Fig. 6(A) shows that the average amount of feedback given to the system after the first task drops, with the majority of the users satisfied with the results of generalization after the initial demonstration (we count the training step in Task 1 as feedback). This result is also reflected in Fig. 6(B), showing

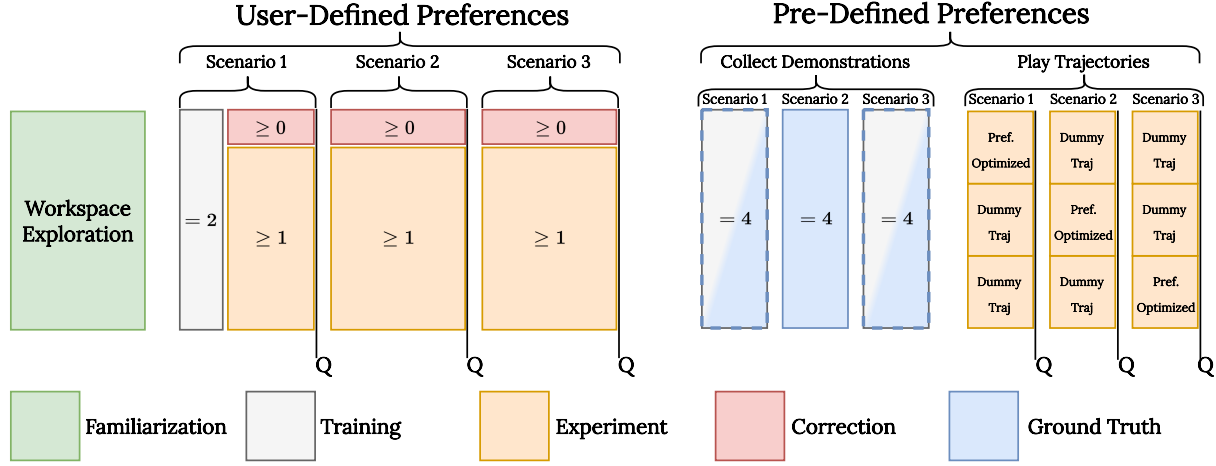


Fig. 5. The experimental protocol. Users started with workspace familiarization, then went through the first experiment assessing the performance of the framework in understanding their preferences. Finally, in the last experiment, they provided ground truth demonstrations and evaluated the demonstrated trajectories in adhering to the set of predefined preferences. The numbers indicate the number of demonstrations given, either by the human (training/correction/ground truth) or the robot (experiment). The order in which the dummy trajectories were shown to the users was different in every scenario. The ‘Q’ symbols indicate when participants were provided with questionnaires.

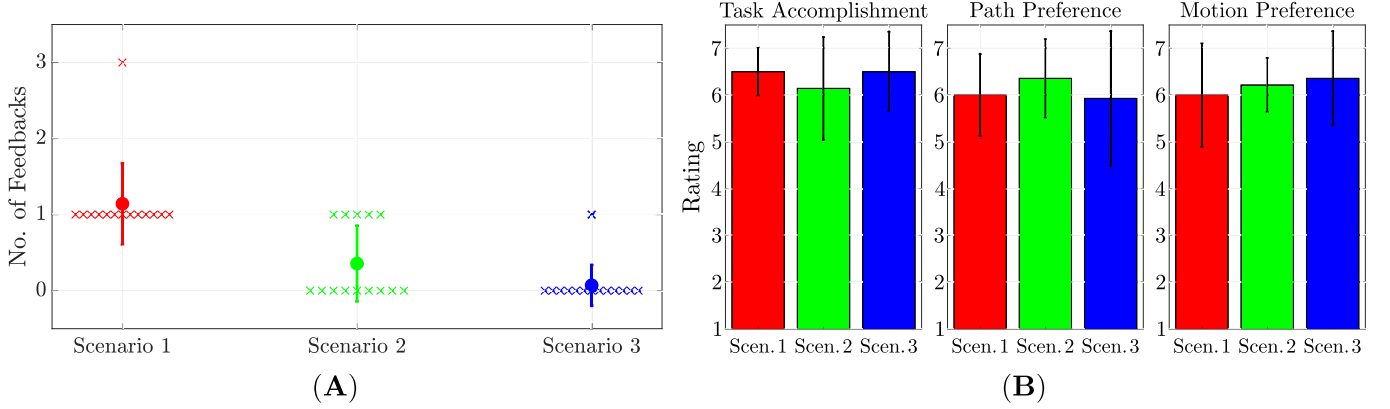


Fig. 6. Results of the first experiment. (A) An average number of feedback provided to the system for each task. The dot represents the mean score, the error bars represent the standard deviation, and the crosses indicate individual data points. (B) Results of the Likert questionnaire for the first resulting trajectory in every task (i.e. prior to any additional demonstrations) - the error bars correspond to standard deviation.

that the users scored the first trajectory produced in every scenario consistently high for all three statements, supporting the claim that the framework can generalize both path and velocity preferences to new task instances. This provides strong evidence in favor of both **H1** and **H2**.

The NASA-TLX results in Fig. 7 show that the users experienced low mental and physical workload. Although kinesthetic teaching is normally associated with high effort, our framework’s effort scores remain mostly on the lower side of the scale. One participant was particularly strict on a height preference the algorithm failed to capture, resulting in 3 iterations of feedback in Scenario 1. Overall, the results in Fig. 7 support **H2**.

### B. Pre-Defined Preferences

To objectively evaluate the accuracy, and the user’s ability to discern preferences, we conduct an experiment where users are asked to adhere to the following path preferences (we did not consider velocity preferences in this experiment):

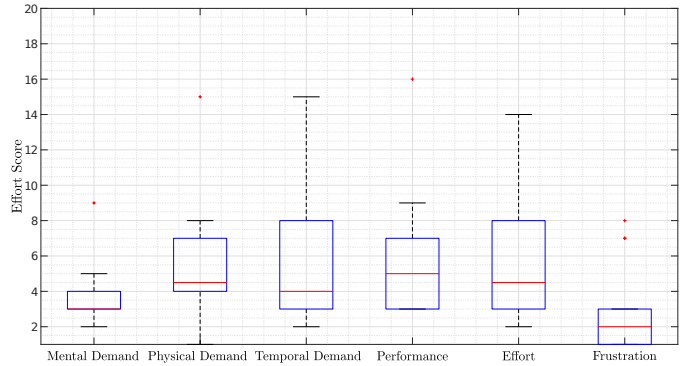


Fig. 7. Results of the NASA-TLX questionnaire after the first experiment.

- Pass on the side of the obstacle that is closer to the robot.
- Stay far from the obstacle.
- Keep a high elevation from the table.

Exactly how to express these preferences, and how to trade

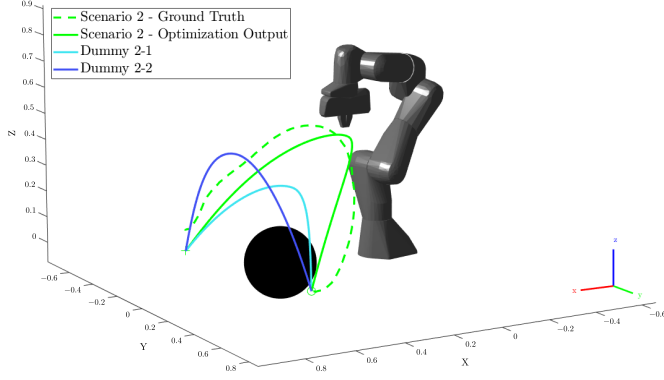


Fig. 8. Scenario 2 results (second experiment) for a single user. The dummy trajectories, in light and dark blue, are designed not to meet the ‘height from table’ and ‘obstacle side’ preferences respectively. The green dashed and solid lines are the mean of human ground truth demonstrations and the robot trajectory respectively. The black sphere represents the obstacle. The framework was trained on data from Scenario 3 and had no access to the ground truth shown.

TABLE I  
AVERAGE DISTANCE ERROR OF TRAJECTORY SAMPLES W.R.T. THE GROUND TRUTH, NORMALIZED W.R.T. DISTANCE OF START TO GOAL, IN METERS: MEAN [MIN, MAX].

|           | Scenario 1        | Scenario 2        | Scenario 3        |
|-----------|-------------------|-------------------|-------------------|
| Optimized | 0.14 [0.09, 0.18] | 0.20 [0.12, 0.27] | 0.17 [0.13, 0.24] |
| Dummy 1   | 0.24 [0.13, 0.34] | 0.26 [0.16, 0.38] | 0.30 [0.21, 0.41] |
| Dummy 2   | 0.27 [0.18, 0.33] | 0.39 [0.28, 0.47] | 0.23 [0.18, 0.28] |

off between them if necessary, is left to the users. We test the following hypotheses:

**H3.** The method remains consistently accurate in all scenarios.

**H4.** Users can clearly distinguish that the output of the framework is following the specified preferences.

**Procedure and Measures.** We collected four demonstrations per scenario. For half of the participants, we trained the model on the mean of the four demonstrations from the first scenario, and for the other half, we used the mean of data from the third scenario. This is to establish that our method generalizes, even when changing the set used as the training data.

After that, the users were shown 3 trajectories per scenario: the output of our framework, and two dummy trajectories (Fig. 8). The dummy trajectories were designed to adhere to 2 out of 3 path preferences. This allowed us to observe if users could distinguish our method’s results compared to sub-optimal trajectories.

As an objective measure of the accuracy of our method, we computed, per scenario, the total Euclidean distance of samples within each trajectory with respect to the mean of the demonstrations (using  $N=80$ ). Furthermore, we compare the total feature counts along each trajectory and measure the error with respect to the ground truth in the feature space.

Subjectively, users rated a 7-point Likert scale per trajectory: “The robot adhered to the demonstrated preferences”.

**Results.** Fig. 8 shows a generalization result of our method under the aforementioned path preferences. The robot attempts to capture and optimize for each user’s personal interpretation

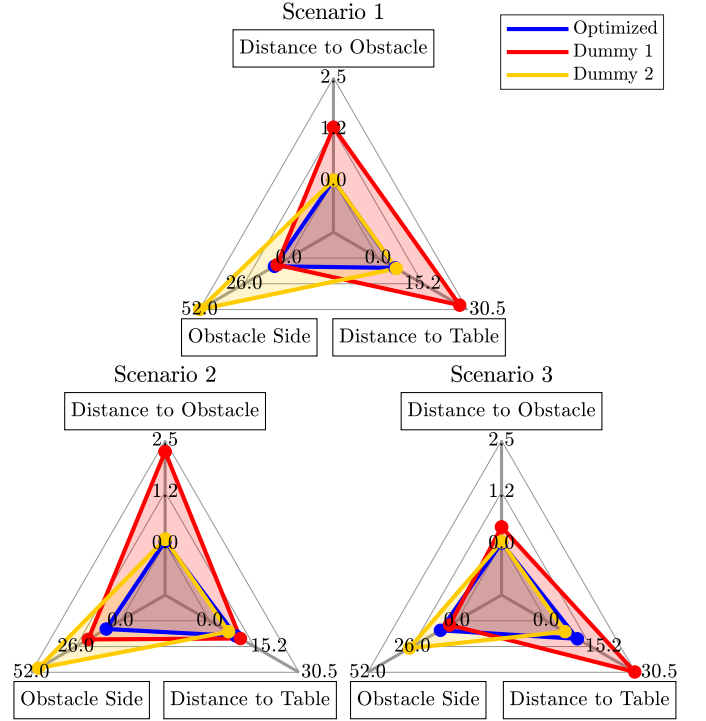


Fig. 9. Total feature count errors of each path preference (all participants), w.r.t. the ground truth (i.e. smaller values for each axis are favored).

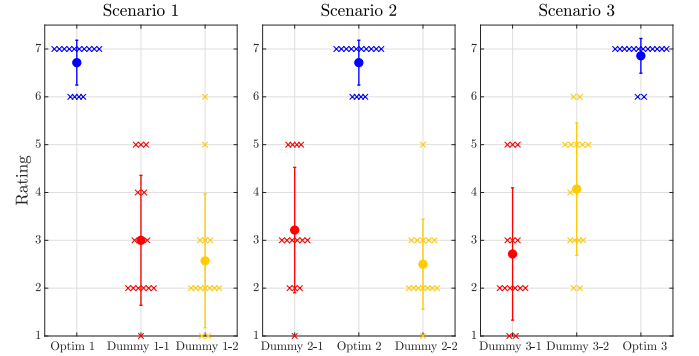


Fig. 10. Result of Likert questionnaire for experiment 2. Crosses indicate individual ratings, while the dot and error bar, respectively, represent mean and standard deviation. Users clearly recognize and highly rate the output of the framework in terms of adhering to path preferences.

of the preferences (e.g. one user’s definition of ‘high’ is different from another). We show the combined results of all users in Tab. I, listing the trajectories’ mean, min and max Euclidean distance to the ground truth, normalized relative to the start-to-goal distance in each scenario (respectively 1.08, 0.74, 0.88 m). The optimized trajectories have the smallest error, but the results only partially support **H3**, as the error in Scenario 2 and 3 is slightly larger than in Scenario 1. This scenario has the longest distance from start to goal, for which the framework seems to perform better.

Fig. 9 shows the errors of the trajectories in feature space. In all scenarios, our optimization result occupies the smallest area. However, in Scenario 2 and 3 the optimized trajectories occupy a slightly larger area than in Scenario 1, showing the



same trend of performance loss in scenarios with the shorter length. Furthermore, in Scenario 2 and 3, dummy trajectories occasionally perform slightly better for one of the preferences. Nevertheless, we see in Fig. 10 that users clearly score the output of our framework higher, which strongly supports **H4**. This indicates that users prefer all preferences to be satisfied simultaneously. The best performing dummy (S3-D2), with the smallest area in Fig. 9 and lowest values in Tab. I, correlates to a high rating in Fig. 10. This also supports **H4**, suggesting that non-expert users intuitively recognize such preferences in trajectories.

### C. Discussion

As the state-of-the-art methods do not have the same functionalities (e.g., path-velocity separation) as the proposed method, we conducted a user study only on the proposed method itself. To account for that, we employed absolute types of metrics (i.e., Likert and NASA-TLX), which can be interpreted independently, rather than tied to a specific external baseline. For example, the Likert scale is tied to an agreement with the given statements and the natural point on the agreement scale serves as a general baseline. The advantage of this is that the result is not tied to a specific relative baseline. If methods that enable the same functionalities are developed in the future, the same Likert scale/questionnaire can be employed to compare the subjective results independently of a specific baseline.

An advantage of the proposed method is that it learns fast. During the first part of the user study, participants spent on average 16.5 s interacting with the robot before expressing satisfaction with the results. This is partially due to having access to kinesthetic demonstrations. This method of demonstration has been criticized as challenging in applications involving high DoF manipulators (Jain et al., 2015; Akgun et al., 2012). However, the separation of learning and control in our framework means that users do not have to provide the correct configuration of the arm in their demonstrations. This feature made it significantly easier for the users to provide demonstrations, which is reflected in the reported low mental and physical loads (Fig. 7).

The separation of path and velocity planning has additional benefits. Formulating the optimization as a multi-objective problem with both position and velocity features results in undesirable interaction of objectives. For instance, when velocity features reward high speeds, the trajectory would converge to a longer path. Conversely, path features with high rewards in specific regions of space would result in slow motion in those regions to increase the density of samples and consequently the overall reward. On the other hand, the separated trajectory optimization has the limitation that it cannot account for dynamical quantities such as joint velocity and acceleration, and the efficiency of movements in robot’s joint space can not be considered.

A challenge with our definition of robot and user objectives is that the trajectory optimization outcome does not always align with task requirements. For instance, a strong “stay close to the object” preference can result in a minimum cost for a

trajectory that is briefly in a collision. Tuning the collision weight can only partially solve this issue, as at a certain point this cost can interfere with the path preferences.

Our user study results showed that non-expert users can intuitively use our method to quickly teach a wide range of preferences to the robot. While the generalization results to different task instances show that we do not always reproduce trajectories with the exact desired shapes in the workspace (see Fig. 8), the subjective performance evaluation shows that users still deem these trajectories highly suitable in terms of task accomplishment and the preferences achieved. State-of-art LfD methods are very capable of producing accurate and complex dynamic movements (Mülling et al., 2013). However, in tasks where there are multiple ways of achieving the same goal, we prefer to trade off motion accuracy for achieving planning propensities on a higher level.

Unfortunately, our approach inherits the limitations of IRL approaches that require specifying reward features by hand. Both features and robot rewards depend on several parameters which require tuning. The problem becomes especially difficult as our features simultaneously govern the behavior of reward learning and trajectory optimization. For instance, high gradients in the feature function lead to erratic behavior of the optimizer, leading to poor solutions and convergence to local optima. Yet, for certain features, a sufficiently high gradient is required to facilitate the learning of preference weights that are large enough to counterbalance each other. As a result, we had to resort to further tuning of parameters, such as the learning rate in (4). An interesting direction for future work would be to test if and how well these issues can be alleviated by feature learning from additional demonstrations, as was done in Bobu et al. (2022). Furthermore, an approach in (Katz et al., 2021) could be employed to learn the relative weighting among features and add additional features through nonlinear functions using neural networks.

In feature engineering or learning, the definition of the context determines how expressive the features are. We considered a limited set of vectors as the context in this work (i.e. obstacle position, start and goal positions). It is possible to include additional information, such as object properties (e.g. sharp, fragile or liquid) (Jain et al., 2015), human position (Bajcsy et al., 2017; Losey et al., 2022), and number of objects. The more rich the context, the more preferences the model can capture in complex environments. However, training diversity can become an issue with contextually rich features, as the model would require more demonstrations to cover a wider range of situations. This will increase training time. An evaluation of the trade-off between improved generalization and higher training time is left for future work.

## IV. SUPPLEMENTARY COMPARISON STUDY

The purpose of this supplementary study is to highlight different aspects and properties of our method in comparison to two common methods from the literature: Dynamic Movement Primitives (DMPs) (Calinon and Lee, 2019) modified with potential fields for obstacle avoidance (Gams et al., 2016), and the method used in Bajcsy et al. (2017) (referred to

as PHI). Since these methods are different conceptually and by design (i.e., optimize for different properties), quantitative comparison is not meaningful. Thus, we examine their aspects in a practical transportation task qualitatively. These aspects are: adherence to preferences, robot objectives, trajectory feasibility, and online learning. In the next subsections, we first discuss the different aspects in more detail, before showing the effects in the transportation task and discussing the pros and cons of the different methods.

#### A. Conceptual differences per aspect

1) *Adherence to preferences*: The methods capture preferences in a different way. Even though we added obstacle awareness to the DMPs we compare to, they lack an explicit notion of preferences. A forcing function is learned to match the shape and velocity distribution of the demonstration, but without any parameterization over features that may capture behavior relative to the context. The potential fields for obstacle avoidance add a basic level of context-awareness, but a predefined one.

Both our method and PHI learn an explicit preference model that is structured as a linear combination of context-parameterized features. Like our model, PHI considers the “*Height from the Table*” and “*Distance to the Obstacle*”. We additionally consider the “*Obstacle Side*”, such that our features cover the different dimensions in space and allow us to capture the preferences in every direction. PHI instead considers other features, such as “*Distance to Human*” and “*Efficiency*”.

Our features are counterbalanced by explicit robot objectives (Sec. IV-A2). In PHI, it is possible to replace the features with the features we use, including the ones for the robot which will not be updated during learning. This way, we can test the effects of the change of features and the change of method.

2) *Robot objectives*: In contrast to PHI, we chose to explicitly separate objectives such as “*Path Efficiency*” and “*Collision Avoidance*”, from the preferences we try to capture. Instead, we let the robot have a reward function of its own. The same effect can be achieved in PHI by fixing the weights of selected features.

The effects of the trade-off between the learned human rewards and the given robot objectives visible in the iterative updates can be viewed as a negotiation between the preferences of two independent agents. We believe that this separation and negotiation will be beneficial especially as tasks become more complex and the artificial agent has knowledge complementary to the human. The benefits may be less visible in the simple task considered in this paper.

As DMPs do not explicitly model an objective function to be optimized, this attribute does not apply.

3) *Trajectory feasibility*: Our method does not automatically check if the planned trajectory is feasible to execute by the robot. A motion feasibility objective can be added to the robot objective function to take this into account in the path optimization.

Rather than weighing the learned preferences against robot objectives, PHI ensures motion feasibility by optimizing the

trajectory in the robot configuration space. This requires an additional simulation step, incorporating the kinematic model of the robot, during trajectory optimization. But then, no corrections need to be applied in hindsight to ensure trajectory feasibility.

4) *Online Learning*: Our method requires a full trajectory to learn from, whereas PHI updates the internal model at each time step. This potentially makes our method less efficient. On the other hand, it allows us to capture velocity preferences in addition to path preferences. Also, because we separate the demonstration from the execution, we obtain a more ‘clean’ observation of the preferences, as we do not have to deduce from the interaction forces what the human demonstration would have looked like without the robot interference. This is likely to benefit generalization. In case of large user corrections, it may even reduce the user’s effort to demonstrate their preferences without the robot interfering. Especially velocity preferences have been found cumbersome to correct in an online manner (Losey and O’Malley, 2019).

When it comes to online updates, DMPs are the fastest, because there is no complex underlying model that needs to be updated. But the trade-off of having a much simpler model is that it lacks the ability to capture preferences in a way that might generalize to changes in the scenario.

All three methods update their model to reduce the error with respect to the latest observation from demonstration. A learning rate trades off learning and overfitting on the corrective demonstration. DMPs updates correct the behavior on the trajectory level, whereas PHI and our method update at a higher level where the observed trajectories are considered a consequence of a human reward model. Nevertheless, future observations that appear contradictory to earlier ones will cause (partial) unlearning of the earlier updates. This results in erroneous behavior learned from imperfect corrections to be corrected, but may in some cases also lead to undesired unlearning.

#### B. Comparison

We will now present a qualitative comparison between the three methods, PHI with two different feature sets:  $_{\phi_{\text{orig}}}$ ,  $_{\phi_{\text{our}}}$ . Our aim is to show the effects of the conceptual differences discussed in Sec. IV-A. To make the comparison as fair as possible, we let all the models learn from the same demonstration data. All methods have access to and consider the obstacle position for planning.

We modify PHI to bypass the estimation of the human desired trajectory from forces, as we have direct access to the desired trajectory from demonstration. We compute the “human correction” every time step from the mismatch between the planned trajectory and the demonstration. The trajectory optimization in PHI requires a robot model for the optimization. As our trajectory optimization does not take the robot dynamics into account, we use a fully actuated point mass for the trajectory optimization. In order to achieve comparable smooth optimal paths, we interpolated the trajectory with a spline instead of linearly as was done originally. For both sets of features, the feature weight ranges and update rates were

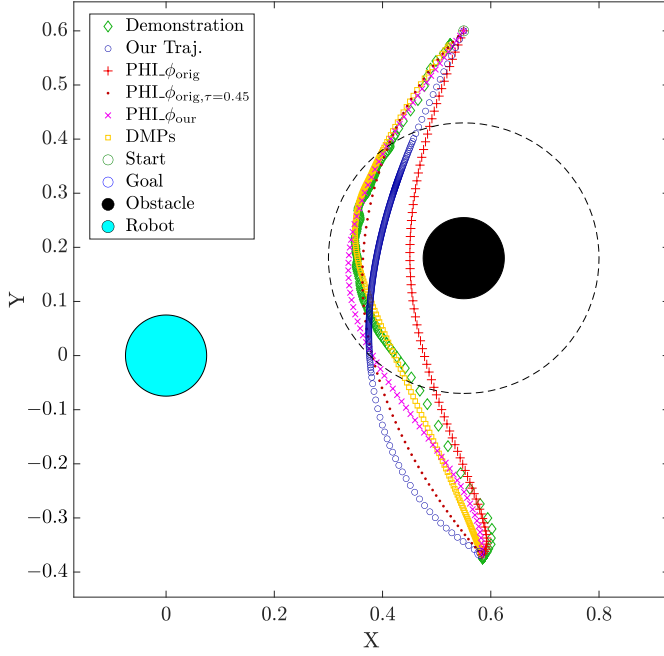


Fig. 11. Training scenario with the human demonstrated trajectory (green diamonds) and the learned reproductions: ours in dark blue circles,  $\text{PHI}_{\phi_{\text{orig}}}$  in red plus signs, with an intermediate learning result in dots,  $\text{PHI}_{\phi_{\text{our}}}$  in purple crosses, and DMP in yellow squares. By placing the markers at equal time intervals, we display the velocity on the trajectories (i.e. the closer the markers, the slower the motion). As PHI does not support differences in velocity, all red and purple markers are spaced equally along the trajectory. The black, cyan, blue, and green circles respectively represent the obstacle, robot, goal (bottom), and start (top) positions. For this study, we set  $d_c = 22.5$  cm (indicated by the dashed circle). We consider points within this region as “close” to the obstacle.

hand-tuned to achieve as close a trajectory match in the initial scenario as we could manage. This initial scenario is illustrated in Fig. 11.

We consider a situation where the user has a preference for “passing on the close side of the obstacle” due to the existence of a wall on the other side that the robot is not aware of. Furthermore, we want to “remain close to the obstacle”, and to “slow down when passing close to the obstacle”. We use a single kinesthetic demonstration containing these three preferences as the input to all methods. For  $\text{PHI}_{\phi_{\text{orig}}}$ , we obtained the correct choice of obstacle side in Fig. 11 by assuming a person standing on the other side of the obstacle and making use of their “human feature”, learning not to come too close to the human.

Fig. 11 shows the demonstration we use for training, as well as the trajectories obtained from the three methods. As the results are generated for the same context as in the demonstration, these results reflect the performance prior to any generalization of preferences. As PHI updates its internal model at every time step, we observe partial unlearning of some features towards the end of the trajectory. This is particularly visible for the “Obstacle Distance” in  $\text{PHI}_{\phi_{\text{orig}}}$ . In Fig. 11, we show an additional trajectory  $\text{PHI}_{\phi_{\text{orig}}, \tau=0.45}$ , which is generated by PHI with the original features and the weights learned at 45% of the trajectory. We see that  $\text{PHI}_{\phi_{\text{orig}}, \tau=0.45}$  is considerably closer to the demonstrated

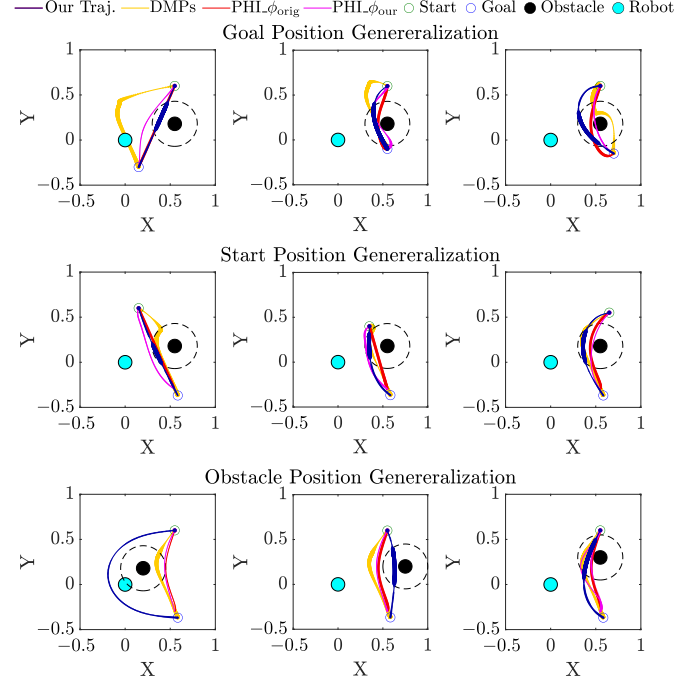


Fig. 12. We demonstrate generalization by modifying the goal (top), start (middle), and obstacle (bottom) positions. The yellow, blue, and red and purple trajectories correspond respectively to the output of the DMPs, our framework, and the two versions of PHI. The thickness of the line indicates the inverse of normalized velocity (i.e. the thicker the line, the slower the trajectory).

trajectory. The demonstrated trajectory has many waypoints close together, quite close to the obstacle, as it slows down when passing it. PHI, on the other hand, has its waypoints equally spaced. As a result, towards the end of the trajectory, a considerable batch of PHI waypoints is further away from the obstacle by default. When the weights continue to update on the difference, we obtain the trajectory  $\text{PHI}_{\phi_{\text{orig}}}$ , which lies closer to the obstacle. With our features, in  $\text{PHI}_{\phi_{\text{our}}}$ , the effect is less pronounced as the features trade off differently, yet the learned path is still different from Our Trajectory, as PHI uses a different trajectory optimization method.

Especially considering  $\text{PHI}_{\phi_{\text{our}}}$ , all three methods perform reasonably well in terms of adhering to the aforementioned path preferences, with a slight variation in how close the robot passes by the obstacle. As discussed in Sec. IV-A, PHI is not able to capture any velocity preferences. Notably, DMP performs well in this aspect as it is able to replicate the demonstrated behavior in terms of both path and velocity.

Next, we modify the scenario nine times, in three different ways: changing respectively the goal, start, and obstacle positions. We compare how each method is able to generalize the initial observation to the different contexts. Fig. 12 displays trajectories produced by the three methods in the nine new scenarios, PHI with the two different feature sets.

We observe that the trajectory by our method (shown in dark blue) passes on the left side of the obstacle, close to the robot, in every case; and the velocity preference, of slowing down when passing close to the obstacle, is only achieved by our framework.

TABLE II  
QUALITATIVE EVALUATION OF THE DIFFERENT ASPECTS OF THE THREE METHODS: DMPs, PHI, AND OURS. THE MARKER ‘O’ INDICATES A VALUE BETWEEN ‘-’ AND ‘+’.

|      | Adherence to preferences <sup>a</sup> | Robot objectives <sup>b</sup> | Trajectory feasibility <sup>c</sup> | Online learning <sup>d</sup> |
|------|---------------------------------------|-------------------------------|-------------------------------------|------------------------------|
| DMPs | -                                     | -                             | -                                   | +                            |
| PHI  | o                                     | o                             | +                                   | o                            |
| Ours | +                                     | +                             | o                                   | -                            |

<sup>a</sup>The criteria, based on Fig. 12, where ‘-’ is given for adherence to only a few, or inconsistently many, preferences, and ‘+’ for adherence to most preferences in most of the cases.

<sup>b</sup>The criteria, based on the model structure, where ‘-’ is given when no robot objectives can be added, and ‘+’ when arbitrary robot objectives can be added.

<sup>c</sup>The criteria where ‘-’ indicates no guarantees for trajectory feasibility, and ‘+’ indicates trajectory feasibility can be guaranteed at all times.

<sup>d</sup>The criteria where ‘-’ indicates the inability to learn in real-time, and ‘+’ indicates the ability to learn and re-plan while the task is being executed.

### C. Discussion

The comparison with DMPs illustrates how a lack of higher-level knowledge about why a trajectory was demonstrated in a specific manner leads to failure in generalization to new contexts. These results emphasize the need for consideration of human models, such as our reward in (1), in LfD methods. PHI, with its model, does considerably better. However, we observe that the internal trajectory optimization reacts differently to the different sets of features, resulting in slight differences in generalized trajectories. The main point, regardless of the applied features, remains that PHI is not able to capture velocity preferences. Tab. II summarizes the strengths and weaknesses of the three methods with respect to the aforementioned aspects.

PHI optimizes the trajectory in the joint space, which can be done fast since inverse kinematics is only required at waypoints. It ensures the planned trajectories are feasible for the robot, which can be interpreted as implicit robot objectives being satisfied. On the other hand, our method optimizes the trajectory in task space, thus additional inverse kinematics computations are necessary together with an explicit description of corresponding robot objectives. The use of inverse kinematics can also be problematic when there are redundant DoF or when there are potential self-collisions. Nevertheless, planning in the task space is closer to where the human preferences typically are (i.e., more intuitive) and can handle obstacle avoidance in a manner that is more predictable for a non-expert human.

It should be noted that our framework does take up to two minutes of optimization (total for path and velocity), whereas the DMPs’ trajectory is produced instantly. However, there is no guarantee that the DMPs will encode and generalize the desired preferences.

## V. CONCLUSION AND FUTURE WORK

We presented a novel approach for learning and executing human preferences in robot object-carrying tasks. Our user study showed fast convergence of the algorithm, and a proof-of-concept for generalizing path and velocity preferences. The efficiency and accuracy of our approach were validated in a real-world scenario. Our supplementary study compares the performance of our framework to two common methods from the literature, providing additional insights into the benefits

and drawbacks caused by the structural differences between the methods. Both in the user study and in the supplementary study, a single informative feedback sufficed (in all cases except one) to capture the human preferences. In the user study, this was tested without prescribing a preference to the users. Our framework was in most cases successful in generalizing these preferences to previously unseen scenarios. Our results support that our model contributes to personalized planning of object-carrying tasks with low interaction effort.

Future studies comparing our method (with just path preferences) to PHI (Bajcsy et al., 2017; Losey et al., 2022) in a user study could lead to useful insights into people’s preferences on iterative versus online learning. Further research could consider a combination of our method and PHI that would benefit from the advantages of both, namely achieving generalization both in-task and over new task instances through learning from online interaction. Next to that, the trajectory model we used to make the problem tractable is quite simplistic and does not describe human motion behavior very well. Future research can aim to replace this model with a library of motion primitives generated from demonstrations to better capture the shape of the trajectories. More accurate trajectory models can enable the extension of the framework to settings where the human and robot come into contact with each other through a shared object (physical human-robot collaboration). Furthermore, it should be studied whether more complex non-linear formulations of the reward function using Gaussian Processes (Bıyık et al., 2020) or Neural Networks (Ibarz et al., 2018), and/or learning them from user input (Bobu et al., 2022; Katz et al., 2021), can effectively capture context-aware preferences without the need for rigorous feature engineering. We believe the presented framework is especially effective in collaborative settings where knowledge of the preferences of a partner is essential to the execution of the task.

### AUTHOR CONTRIBUTIONS

A.A., L.v.d.S., L.P., and J.K. developed the concept and methods. The programming was mainly done by A.A., while L.v.d.S. modified PHI for the supplementary comparison study. A.A. and L.v.d.S. contributed to the experiments and data analysis and wrote the first draft of the paper. L.v.d.S., L.P., and J.K. revised the paper. All authors read and approved the submitted version.



## FUNDING

The work was partially supported by the European Research Council Starting Grant TERI “Teaching Robots Interactively” (project reference 804907) and European Space Agency through the project “Rhizome: Off-Earth Manufacturing and Construction”. This study received funding from the Honda Research Institute Europe.

## INSTITUTIONAL REVIEW BOARD STATEMENT

The study was conducted in accordance with the Declaration of Helsinki, and approved by the Human Research Ethics Committee at the Delft University of Technology on 06/09/2021.

## INFORMED CONSENT STATEMENT

Informed consent was obtained from all subjects involved in the study.

## DATA AVAILABILITY STATEMENT

The datasets generated for the performed user and supplementary study are available on request. Please contact the corresponding author.

## ACKNOWLEDGMENTS

The authors thank Andreea Bobu for her feedback on the paper.

## CONFLICTS OF INTEREST:

The authors declare no conflict of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

## REFERENCES

- Akgun, B., Cakmak, M., Jiang, K., and Thomaz, A. L. (2012). Keyframe-based learning from demonstration. *International Journal of Social Robotics*, 4(4):343–355.
- Bajcsy, A., Losey, D. P., O’Malley, M. K., and Dragan, A. D. (2017). Learning robot objectives from physical human interaction. In *Conference on Robot Learning*.
- Bıyık, E., Huynh, N., Kochenderfer, M. J., and Sadigh, D. (2020). Active preference-based Gaussian process regression for reward learning. *arXiv preprint arXiv:2005.02575*.
- Bıyık, E., Losey, D. P., Palan, M., Landolfi, N. C., Shevchuk, G., and Sadigh, D. (2022). Learning reward functions from diverse sources of human feedback: Optimally integrating demonstrations and preferences. *The International Journal of Robotics Research*, 41(1):45–67.
- Bobu, A., Wiggert, M., Tomlin, C., and Dragan, A. D. (2022). Inducing structure in reward learning by learning features. *The International Journal of Robotics Research*, page 02783649221078031.
- Calinon, S. and Lee, D. (2019). Learning control. In Vadakkepat, P. and Goswami, A., editors, *Humanoid Robotics: a Reference*, pages 1–52. Springer.
- Duchaine, V. and Gosselin, C. M. (2007). General model of human-robot cooperation using a novel velocity based variable impedance control. In *Second Joint EuroHaptics Conference and Symp. on Haptic Interfaces for Virtual Environment and Teleoperator Systems*.
- Ewerton, M., Maeda, G., Kollegger, G., Wiemeyer, J., and Peters, J. (2016). Incremental imitation learning of context-dependent motor skills. In *2016 IEEE-RAS 16th International Conference on Humanoid Robots (Humanoids)*, pages 351–358. IEEE.
- Fahad, M., Chen, Z., and Guo, Y. (2018). Learning how pedestrians navigate: A deep inverse reinforcement learning approach. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*.
- Gams, A., Petrič, T., Do, M., Nemec, B., Morimoto, J., Asfour, T., and Ude, A. (2016). Adaptation and coaching of periodic motion primitives through physical and visual interaction. *Robotics and Autonomous Systems*, 75:340–351.
- Hart, S. G. and Staveland, L. E. (1988). Development of NASA-TLX (task load index): Results of empirical and theoretical research. In *Advances in Psychology*, volume 52, pages 139–183. Elsevier.
- Ibarz, B., Leike, J., Pohlen, T., Irving, G., Legg, S., and Amodei, D. (2018). Reward learning from human preferences and demonstrations in Atari. *arXiv preprint arXiv:1811.06521*.
- Ijspeert, A. J., Nakanishi, J., and Schaal, S. (2002). Movement imitation with nonlinear dynamical systems in humanoid robots. In *IEEE International Conference on Robotics and Automation*.
- Jain, A., Sharma, S., Joachims, T., and Saxena, A. (2015). Learning preferences for manipulation tasks from online coactive feedback. *The International Journal of Robotics Research*, 34(10):1296–1313.
- Jeon, H. J., Milli, S., and Dragan, A. (2020). Reward-rational (implicit) choice: A unifying formalism for reward learning. *Advances in Neural Information Processing Systems*, 33:4415–4426.
- Katz, S. M., Maleki, A., Bıyık, E., and Kochenderfer, M. J. (2021). Preference-based learning of reward function features. *arXiv preprint arXiv:2103.02727*.
- Kirby, R., Simmons, R., and Forlizzi, J. (2009). Companion: A constraint-optimizing method for person-acceptable navigation. In *18th IEEE International Symp. on Robot and Human Interactive Communication*.
- Kretschmar, H., Spies, M., Sprunk, C., and Burgard, W. (2016). Socially compliant mobile robot navigation via inverse reinforcement learning. *The International Journal of Robotics Research*, 35(11):1289–1307.
- Kulvicius, T., Biehl, M., Aein, M. J., Tamosiunaite, M., and Wörgötter, F. (2013). Interaction learning for dynamic movement primitives used in cooperative robotic tasks. *Robotics and Autonomous Systems*, 61(12):1450–1459.
- Losey, D. P., Bajcsy, A., O’Malley, M. K., and Dragan, A. D. (2022). Physical interaction as communication: Learning robot objectives online from human corrections. *The International Journal of Robotics Research*, 41(1):20–44.
- Losey, D. P. and O’Malley, M. K. (2019). Learning the correct

- robot trajectory in real-time from physical human interactions. *ACM Transactions on Human-Robot Interaction (THRI)*, 9(1):1–19.
- MathWorks (2018). Waypoint trajectory generator. <https://nl.mathworks.com/help/fusion/ref/waypointtrajectory-system-object.html>. Accessed on 10/08/2021.
- Mülling, K., Kober, J., Kroemer, O., and Peters, J. (2013). Learning to select and generalize striking movements in robot table tennis. *The International Journal of Robotics Research*, 32(3):263–279.
- Nemec, B., Likar, N., Gams, A., and Ude, A. (2018). Human robot cooperation with compliance adaptation along the motion trajectory. *Autonomous Robots*, 42(5):1023–1035.
- Palan, M., Landolfi, N. C., Shevchuk, G., and Sadigh, D. (2019). Learning reward functions by integrating human demonstrations and preferences. *arXiv preprint arXiv:1906.08928*.
- Peternel, L., Petrič, T., Oztop, E., and Babič, J. (2014). Teaching robots to cooperate with humans in dynamic manipulation tasks based on multi-modal human-in-the-loop approach. *Autonomous Robots*, 36(1):123–136.
- Ratliff, N. D., Bagnell, J. A., and Zinkevich, M. A. (2006). Maximum margin planning. In *23rd International Conference on Machine Learning*.
- Shivaswamy, P. and Joachims, T. (2015). Coactive learning. *Journal of Artificial Intelligence Research*, 53:1–40.
- Vasquez, D., Okal, B., and Arras, K. O. (2014). Inverse reinforcement learning algorithms and features for robot navigation in crowds: an experimental comparison. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*.
- Wirth, C., Akrou, R., Neumann, G., and Fürnkranz, J. (2017). A survey of preference-based reinforcement learning methods. *Journal of Machine Learning Research*, 18(136):1–46.
- Ziebart, B. D., Maas, A. L., Bagnell, J. A., and Dey, A. K. (2008). Maximum entropy inverse reinforcement learning. In *AAAI*.
- Zucker, M., Ratliff, N., Dragan, A. D., Pivtoraiko, M., Klingensmith, M., Dellin, C. M., Bagnell, J. A., and Srinivasa, S. S. (2013). CHOMP: Covariant Hamiltonian optimization for motion planning. *The International Journal of Robotics Research*, 32(9-10):1164–1193.